

Arrival_Cleaning

Carole Shaw

2024-11-15

Purpose

This script is designed to put arrival data together and to distinguish data collected for treatment prior to FHCC arrival to arrivals associated with FHCC care. The plan is to make this a function that is called from `icicle_functions.Rmd` so that we will do this consistently over time.

Background:

Currently patient treatment plans each generate a new arrival. Usually those are entered in sequential order, and for those that have had their first treatment here the arrival 1 form contains their first treatment here for AML, either frank disease or >10% blasts. On those records the arrival table is `arrival1`, the `arrivalnum` is also 1, and the treatment location is FHCC.

Some patients first were treated at an outside location. There are a few ways that an outside treatment may have been entered into our system. 1) Induction may have been entered into the Diagnosis REDCap, which is now defunct, but induction associated was imported into the 'induction' form of the Estey List REDCap project. 2) An arrival may have been entered as `arrivalnum` "-1" to indicate an arrival that occurred prior to arrival at FHCC, or 3) an arrival may be entered with location of treatment "OUT" to indicate treatment from an outside location, often (but not always), indicating treatment prior to arrival at FHCC.

We formerly did not enter outside treatment into arrival forms, when that was the case location was implied to be FHCC. The default for location is therefore FHCC when no location is given.

There are some cases where an outside arrival is dated *after* the first FHCC arrival. When that is not associated with hospice/palliative/no treatment those cases should be set aside for review.

If corrections are made to data in REDCap, the REDCap data need to be imported anew for corrections to be reflected here.

Plan

So the logic in this program is:

- 1) Identify any treatment imported from the defunct Diagnosis REDCap into the induction table (**done**)
- 2) Separate arrivals outside treatments into `allprior` table (**done**)
- 3) Separate arrivals represent a treatment at FHCC or without a treatment location into `allarrival` table (**done**)
- 4) Merge arrivals identified from the induction table with those from the `allprior`, remove duplicates (**done**)
- 5) From the `allarrival` table, find the earliest arrival for each patient into a table `first_fhcc_arrival` (**done**)
- 6) Compare `allprior` arrivals to the `first_fhcc_arrival` for each patient to find those that need to be set aside for review, ignoring any where the treatment was No Treatment, Hospice or Palliative. (**done**)

- 7) If data cleaning to the REDCap instance occurred at step 6, update REDCap data on SQL server and then re-run steps 1-6, otherwise go on to step 8. (**not yet - easy**)
- 8) Put together the allarrival table with those remaining in the allprior table. (**done**)
- 9) Assign sequence to patient arrivals ordering by arrivaldate, treatmentdate. Using treatment date because many of the outside treatments don't have a concept of arrival. (**not yet - easy**)
- 10) Convert this script into a function within the icicle_functions.Rmd replacing the current function: **get_arrivals(tblcnt=11)**
- 11) Run the script: Coding and Data Acquisition/R Scripts/Frozen to SQLServer/**Frozen_Data_to_SQLServer.Rmd** to update the frozen data. The script exists, and won't need changed if the new function get_arrivals() replaces the old function get_arrivals(). (ready)

SETUP

Load Libraries and Icicle Functions Most libraries are loaded with icicles. Recompile the icicle functions every time to incorporate changes.

```
codedir <- gsub("\\\\", "/", Sys.getenv('FROZEN_PATH', unset = NA))
source(paste0(codedir, '/icicle_setup.R')); icicle_setup()
```

```
## icicle_functions.R created 2024-11-15 15:04:18.505396
```

Helper Functions

convert_var(var_name, reference_var_name, data) Converts a variable to the type of another reference variable and returns a variable with the same values as **var**, but converted to the type of **reference_var**. Conversion uses standard R functions like **as.Date()**, **as.POSIXct()**, **factor()**, and others. The function handles a variety of common R types, such as **Date**, **POSIXct**, **factor**, **numeric**, **character**, **logical**, and others.

- **Date** and **POSIXct** types with **as.Date()** and **as.POSIXct()** conversions.
- **Factor** columns with matching levels to preserve factor levels.
- **Logical**, **integer**, **numeric**, and **character** types using **as.logical()**, **as.integer()**, **as.numeric()**, and **as.character()**.
- For other types, it defaults to using **as()** to match the reference type.

Arguments

- **var**: A variable (or column) whose type needs to be converted. This can be any type that is compatible with R's basic types.
- **reference_var**: A reference variable from which the target type will be determined. This is typically a column from another data frame defining target type.
- **data** is the data frame containing the columns.

```
# Define the custom conversion function
convert_var <- function(var, reference_var) {
  ref_class <- class(reference_var)

  if ("Date" %in% ref_class) {
    return(as.Date(var)) # Convert to Date if reference is Date
  } else if ("POSIXct" %in% ref_class) {
```

```

    return(as.POSIXct(var)) # Convert to POSIXct if reference is POSIXct
  } else if ("factor" %in% ref_class) {
    return(factor(var, levels = levels(reference_var))) # Convert to factor with matching levels
  } else if ("logical" %in% ref_class) {
    return(as.logical(var))
  } else if ("integer" %in% ref_class) {
    return(as.integer(var))
  } else if ("numeric" %in% ref_class) {
    return(as.numeric(var))
  } else if ("character" %in% ref_class) {
    return(as.character(var))
  } else {
    # Handle unsupported types or fallback to standard conversion
    ref_type <- typeof(reference_var)
    return(as(var, ref_type)) # Fallback conversion based on reference's type
  }
}

```

Load Some Data from SQL Server

```

# columns of interest
patient_list_cols <- c("recordid",      "ptmrn",      "ptlastname", "ptbirthdate", "dxdate",
                      "dxmorphdate",    "dxmorphsrc",  "dxmorph",   "dxmorphnum",  "dxmorphcell",
                      "dxflowdate",     "dxflowsrc",   "dxflow",    "dxflownum",
                      "uploaded_to_ctms", "uploaded_to_ctms_date", "firstarrival",
                      "firstinduction",  "patient_list_rectime", "primary_subject_id")

specialty_note_cols <- c("recordid", "ptmrn", "ptlastname",
                        "ptdeathdate", "ptlastchecked",
                        "lastknownalive", "lastknownremission", "lastknownactivedisease",
                        "specialty_note_rectime")

# tables of interest
patient_list <- sql_get_table('esteylist_label.patient_list') |>
  filter(ptlastname != 'PATIENT') |>
  mutate(recordid = as.character(recordid)) |>
  select(any_of(patient_list_cols))

specialty_note <- sql_get_table('esteylist_label.specialty_note') |>
  filter(ptlastname != 'PATIENT') |>
  mutate(recordid = as.character(recordid)) |>
  select(any_of(specialty_note_cols))

# Make sure the REDCap data dictionary is loaded for the current section's schema
get_rc_dictionary('esteylist')

```

Pre-Arrivals from the Induction Table

```

# previous to first arrival induction information
induction_cols <- c("recordid",
  "ptmrn",
  "ptlastname",
  "arrivaltable",
  "arrival",
  "rxline",
  "treatmentlocation",
  "treatmentdate",
  "treatment",
  "responsedate",
  "response",
  "mrd",
  "relapsedate",
  "relapsedesc")

induction <- sql_get_table(glue('esteylist_label.induction')) |>
  # Apply the custom date conversion function across character columns
  convert_date_cols() |>
  # Apply the custom grouping of REDCap checkbox fields into a single column
  group_rc_chkbox() |>
  mutate(recordid      = as.character(recordid)
    , arrivaltable      = "induction"
    , orig_arrivaltable = "induction"
    , arrival           = as.integer(-1)
    , rxline            = 'Ind'
    # , arrivaldate      = dxdate1
    , treatmentlocation = 'OUT'
    , treatmentdate     = dxtreatmentdate1
    , treatment         = dxtreatment1
    , responsedate      = dxresponsedate1
    , response          = dxresponse1
    , mrd               = dxmrd1
    , relapsedate       = inductionrelapsedate
    , relapse           = inductionrelapsenote
    , treatmentdate     = as.Date(treatmentdate)
  ) |>
  select(any_of(induction_cols)) |>
  left_join(patient_list, by = join_by(recordid, ptmrn, ptlastname)) |>
  left_join(specialty_note, by = join_by(recordid, ptmrn, ptlastname)) |>
  filter(!is.na(treatmentdate)) |>
  arrange(ptmrn, treatmentdate) |>
  select(recordid, ptmrn, ptlastname, treatmentdate, arrivaltable, arrival, everything())

```

Arrivals from the tables Arrival1 - Arrival11

```

allprior   = NULL
allarrival = NULL
allnotreat = NULL

tblcnt = 11

```

```

pb <- get_progress_bar(tblcnt, "PROCESS ARRIVALS")

for (n in 1:tblcnt) {
  # table for this loop
  tbl <- paste0('arrival', n)

  # Apply the function to the arrival dataframe
  arrival <- sql_get_table(glue('esteylist_label.arrival{n}')) |>
  # remove test patients
  filter(ptlastname != 'PATIENT') |>
  mutate(
    # Convert recordid to character type
    recordid = as.character(recordid)
    # Add a row number
    , row_number = row_number(), .before = 1
    # Add a tablename column
    , arrivaltable = tbl
    , orig_arrivaltable = tbl
  ) |>
  # Apply the custom date conversion function across character columns
  convert_date_cols() |>
  # Apply the custom grouping of REDCap checkbox fields into a single column
  group_rc_chkbx() |>
  # Apply multiple renames in one `rename_with`
  rename_with(~ {
    col_name <- .
    # Rename "a{n}" fields to "a_"
    col_name <- gsub(glue("^a{n}"), "a_", col_name)
    # Rename "arrival{n}note" fields specifically to "arrival_note"
    col_name <- gsub(glue("^arrival{n}note$"), "arrival_note", col_name)
    # Rename "arrival{n}" fields to "arrival"
    col_name <- gsub(glue("^arrival{n}"), "arrival", col_name)
    # Remove trailing "{n}" from other fields
    col_name <- gsub(glue("{n}$"), "", col_name)
    # Return the transformed column name
    col_name
  }) |>
  mutate(# using regular expressions to flag those that didn't get curative treatment
    , currativerx = !grepl("(?i)hosp|pal|comfort|support|surv|no\\streat", treatment)) |>
  select(row_number, recordid, ptmrn, ptlastname, arrivaldate, arrival_rectime, arrivaltable
    , treatment, currativerx, everything())

arrival <- arrival |>
  left_join(patient_list, by = join_by(recordid, ptmrn, ptlastname)) |>
  left_join(specialty_note, by = join_by(recordid, ptmrn, ptlastname)) |>
  select(row_number, recordid, ptmrn, ptlastname, primary_subject_id, everything())

if (n > 1) {
  # Ensure previous column types match
  arrival <- arrival |>
  mutate(across(
    .cols = intersect(names(arrival), names(allarrival)), # Only columns in both dataframes

```

```

        .fns = ~ convert_var(.x, allarrival[[cur_column()]]) # Pass `.x` and reference column to `conv
    ))
}

# identify rows not associated with a treatment
notreat <- arrival |> filter(str_detect(treatment, '(?i)consult|opinion|surveillance|2nd\\s|(no)+t?\\s'))
# removed rows where there was no treatment
arrival <- anti_join(arrival, notreat, by='row_number')

# combine, and make sure to remove dups
prior <- bind_rows(lapply(ls(pattern = "^prior_"), get)) # prior <- bind_rows(prior, prior1, prior2, ,
prior <- distinct(prior)

# remove prior treatments from the arrival data
prior_1 <- arrival |> filter(currativerx & rxline == 'Treatment prior to AML induction')
prior_2 <- arrival |> filter(currativerx & a_citreatmentlocation == 'OUT' & a_treatmentlocation == 'OUT')
prior_3 <- arrival |> filter(currativerx & a_treatmentlocation == 'OUT')

# combine, and make sure to remove dups
prior <- bind_rows(lapply(ls(pattern = "^prior_"), get)) # prior <- bind_rows(prior, prior1, prior2, ,
prior <- distinct(prior)

# Remove prior treatment data from the arrival data
arrival <- anti_join(arrival, prior, by=join_by(row_number))

# set empty structures if needed, usually in just the first loop, these empty structures should be th
if (is.null(allprior) & is.null(allarrival)) {
  # empty prior structure
  allprior <- as.data.frame(prior)[0,]
  # empty arrival structure
  allarrival <- as.data.frame(arrival)[0,]
  # empty structure for no treatment
  allnotreat <- as.data.frame(notreat)
}

# Join arrivals
allarrival <- bind_rows(allarrival, arrival) |> mutate(row_number = row_number())

# Join priors
allprior <- bind_rows(allprior, prior) |> mutate(row_number = row_number() + nrow(allarrival))

# Join no treatment
allnotreat <- bind_rows(allnotreat, notreat) |> mutate(row_number = row_number() + nrow(allarrival))

# clear out tables from last loop
rm(list = ls(pattern = "^prior"))
rm(list = ls(pattern = "^arrival"))

pb$tick()
}

```

```
rm(pb)
```

Merge Induction arrivals with Arrival 1-11 Prior arrivals

Merge induction data originally from the induction table (originally from Diagnosis REDCap) with allprior arrivals found in arrivals 1-11.

```
# Wonder if we should put bind these earlier on
allprior$arrivaltable = 'prearrival'
allprior <- bind_rows(allprior, induction) |>
  arrange(recordid, arrivaldate, treatmentdate, arrivalnum) |>
  # Add a unique arrival identifier as row number
  mutate(row_number = row_number(), .before = 1)
```

Identify first arrival at FHCC

```
# identify the earliest arrival in the 'allarrival' df
first_fhcc_arrival <- allarrival |>
  mutate(min_date = pmin(arrivaldate, treatmentdate, na.rm = TRUE)) |>
  select(recordid, min_date) |>
  filter(!is.na(min_date)) |>
  group_by(recordid) |>
  summarise(firstarrivaldate = min(min_date) # Get min_date for each group,
    , .groups = 'drop')

allarrival <- left_join(allarrival, first_fhcc_arrival)
```

ERROR CHECK

OUTSIDE treatments dated *before* the first arrival recorded at FHCC

Answers the question: Are there arrivals dated after first FHCC arrival/treatment that we have indicated as “OUT” for location?

Then produces an excel document in the scrap directory with records that seem fishy.

```
# Are there prior arrivals dated after first FHCC arrival/treatment

# Have staff review any of these where arrival date is after first FHCC arrival, but Rx date is not
late_outside_arrival <- inner_join(allprior, first_fhcc_arrival, by = join_by(recordid)) |>
  mutate( priorarrivaldate = arrivaldate
    , priortreatmentdate = treatmentdate
    , afterfirstarrival = priorarrivaldate > firstarrivaldate
    , afterfirsttreatment = priortreatmentdate > firstarrivaldate
    , reviewreason = paste('Arrival date', arrivaldate
      , 'is after first arrival on', firstarrivaldate)) |>
  filter( afterfirstarrival & !afterfirsttreatment) |>
  select( recordid, ptmrn, ptlastname
    , priorarrivaldate, priortreatmentdate, firstarrivaldate)
```

```

# Have staff review any of these where treatment date is after first FHCC arrival, but Rx arrival date
late_outside_treatment <- inner_join(allprior, first_fhcc_arrival, by = join_by(recordid)) |>
  mutate( priorarrivaldate = arrivaldate
    , priortreatmentdate = treatmentdate
    , afterfirstarrival = priorarrivaldate > firstarrivaldate
    , afterfirsttreatment = priortreatmentdate > firstarrivaldate
    , reviewreason = paste('Outside arrival on',priorarrivaldate,
                          'is before first arrival on',firstarrivaldate,
                          'but associated rx on',priortreatmentdate,'is not.')) |>
  filter( afterfirsttreatment & !afterfirstarrival) |>
  select( recordid, ptmrn, ptlastname, reviewreason
    , priorarrivaldate, priortreatmentdate, firstarrivaldate)

# Discrepancy report
outside_rx_discreps <- bind_rows(late_outside_arrival, late_outside_treatment)
rm(list = ls(pattern = "^late_outside_"))

if (nrow(outside_rx_discreps) > 0){
  # Create excel for data cleaning
  make_sheet(outside_rx_discreps, scrapdir, 'errordata', 'outside_after_FHCC_arrival', 'xlsx', overwrite = TRUE)
  # Add to table of things to correct
  addrow('keep_df', scrapdir, 'outside_after_FHCC_arrival.xlsx', 'Review for arrival date, treatment date')
}

```

Merge prior arrivals with FHCC arrivals

```

# Merge together all arrival from FHCC and all prior to arrival at FHCC
allarrival <- bind_rows(allarrival, allprior) |>
  arrange(recordid, arrivaldate, treatmentdate, arrivalnum, arrivaltable) |>
  mutate(row_number = row_number())

```

Rank Patient Arrivals with prev/next

```

# Adding prev/next arrival information}
# Ranking arrivals
allarrival <- allarrival |>
  mutate(arrival_old = arrival) |>
  arrange(recordid, ptmrn, pmax(arrivaldate, treatmentdate, na.rm = TRUE)) |>
  group_by(recordid) |>
  mutate(
    prevarrivaldate = lag(arrivaldate),
    prevarrivaltype = lag(arrivaltype),
    prevarrivalreason = lag(arrivalreason),
    prevarrivalmorph = lag(arrivalmorph),
    prevarrivalflow = lead(arrivalflow),

    nextarrivaldate = lead(arrivaldate),
    nextarrivaltype = lead(arrivaltype),
    nextarrivalreason = lead(arrivalreason),
  )

```



```

nextarrivalmorph = lead(arrivalmorph),
nextarrivalflow  = lead(arrivalflow),

# Adding ranking
arrival = row_number(),

# Count all treatment plans
allarrivalcnt = n(),

# Count FHCC arrivals
fhccarrivalcnt = sum(grepl("^arr", arrivaltable))

) |>
ungroup() |>
select(recordid, ptmrn, arrival, everything())

```

Report Missing Arrival Sequence number or Arrival Date

Very hard to get arrivals in order without this information, luckily not very many are missing these.

```

# identify errors
missing_arrival_order_info <- allarrival |>
# bind_rows(allnotreat) |>
left_join(patient_list |> select(recordid, patient_list_rectime)) |>
arrange(recordid, arrivaldate, treatmentdate, arrivalnum, arrivaltable) |>
filter(is.na(arrivaldate) | arrivalnum == "") |>
filter(recordid > 1000 & arrivaltable != 'induction') |>
arrange(ptlastname, ptmrn, arrivalnum) |>
select( recordid, ptmrn, ptlastname
        , arrivalnum, arrivaltable, orig_arrivaltable, arrivaldate, arrival_rectime, arrivalnote
        , treatmentdate, treatmentnote, patient_list_rectime )

if (nrow(missing_arrival_order_info) > 0){
  # Create excel for data cleaning
  make_sheet(missing_arrival_order_info, scrapdir, 'nodate_or_nonumber', 'missing_arrival_order_info',
  # Add to table of things to correct
  addrow('keep_df', scrapdir, 'missing_arrival_order_info.xlsx', 'Review for arrival date missing and a
}

```

MAP KEY VARIABLES, SAVE TO SQLSERVER, PATIENTS W/O ARRIVALS

These are patients who are 'orphaned' in our system, they have been added to REDCap, but have not yet had arrival information entered

```

allarrival <- treatment_mapping('allarrival')
allarrival <- response_mapping('allarrival')

allpats    <- bind_rows(allarrival, allnotreat)

```

```

orphan_pats <- anti_join(patient_list |>
  select(recordid, ptmrn, ptlastname), allpats, by=join_by(recordid, ptmrn, ptlastname)) |>
  mutate(arrivaldate=NA_Date_, arrivaltable='prearrivalentry')

rm(allpats)

allarrival <- bind_rows(allarrival, orphan_pats) |>
  mutate(arrivalyyyymm = format(arrivaldate, '%Y%m'))

sql_insert_df(allarrival, 'frozen.allarrival', nrow=0)
sql_insert_df(allarrival, 'frozen.allarrival', date_suffix='%Y%m', nrow=0)

make_sheet(orphan_pats, scrapdir, 'Orphans', 'orphan_redcap', 'csv', date=TRUE)
addrow('keep_df',
  scrapdir,
  'orphan_redcap.csv',
  paste(nrow(orphan_pats), 'Patients added to Estey List but not yet abstracted\n'))

cat(nrow(orphan_pats), 'Patients added to Estey List but not yet abstracted\n')

```